



Detecting Fraud in Online Transactions: A Machine Learning Approach

by

Nguyen Quoc Trong Nghia

Submitted to the School of Science, Engineering and Technology
in partial fulfilment of the requirements for the degree of

Master of Artificial Intelligence

at the

RMIT UNIVERSITY

September 2026

© Nguyen Quoc Trong Nghia, 2026. All rights reserved.

Certified by

Dr. Bao Nguyen
RMIT University, Thesis Supervisor

Accepted by

Dr. Thuy Nguyen, Course Coordinator
School of Science, Engineering and Technology

Declaration

I certify that except where due acknowledgement has been made, this thesis is my own work, that any help received in preparing this thesis has been acknowledged, and that all sources used have been indicated. I certify that the work is original and has not been submitted previously, in whole or in part, for any other academic award. Any use of generative AI tools in the preparation of this thesis has been disclosed in accordance with course policy.

Student name: Nguyen Quoc Trong Nghia

Student number: s3343711

Signature: Nguyen Quoc Trong Nghia

Date: 15/09/2026.....

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Dr. Bao Nguyen, for his guidance, valuable feedback, and support throughout this research. I would also like to thank Dr. Thuy Nguyen, the course coordinator, for her support and assistance throughout the semester.

Finally, I gratefully acknowledge RMIT University Vietnam and the School of Science, Engineering and Technology for providing the resources and academic environment that supported the completion of this research.

Generative AI tools, primarily ChatGPT by OpenAI, were used as a supporting tool during the preparation of this thesis. Their use included improving language clarity and academic phrasing, reviewing document structure and consistency, identifying repetition, and discussing the presentation and interpretation of experimental findings.

The experimental design, implementation, model training, hyperparameter optimization, evaluation, and final interpretation of the results were undertaken by the author. AI generated suggestions, factual claims, and references were reviewed before inclusion. The author takes full responsibility for the accuracy and final content of this thesis.

Summary

Fraudulent transaction detection is difficult because fraudulent transactions are uncommon and transaction behavior can change over time. This research examines how different approaches to preparing transaction data, handling the imbalance between fraudulent and legitimate transactions, and selecting prediction models affect fraud detection.

The study uses the IEEE-CIS Fraud Detection dataset and compares six machine learning models. Models are trained using earlier transactions and tested using later transactions. Their performance is also compared using hidden test data from the same public benchmark.

Creating additional transaction information did not consistently improve the results. Removing sparse, redundant, and weakly useful information reduced the number of inputs while maintaining similar overall performance. Different prediction models also performed better under different evaluation criteria.

Methods for handling the imbalance between fraudulent and legitimate transactions produced different trade offs. Some configurations detected more fraudulent transactions but incorrectly flagged more legitimate transactions, while others reduced these false alerts but missed more fraud. Overall, no single approach was best in every situation, and the preferred approach depends on the evaluation objective.

Abstract

Fraud detection in online transactions is challenging because fraudulent transactions are rare and transaction patterns can change over time. This study evaluates how feature representation, feature reduction, model selection, and class imbalance handling affect fraud detection performance using the IEEE-CIS Fraud Detection dataset.

Six supervised machine learning models were evaluated across 72 experimental configurations using a chronological hold out test set and the hidden Kaggle benchmark. Performance was assessed using ROC-AUC, PR-AUC, Precision, Recall, F1-score, Balanced Accuracy, and MCC.

Engineered entity and temporal features produced limited aggregate improvement. Feature reduction removed approximately one fifth of the predictors while maintaining similar mean local ROC-AUC and producing higher mean private Kaggle ROC-AUC. LightGBM achieved the highest mean local ROC-AUC, while XGBoost performed best across several minority class metrics. CatBoost achieved the highest mean public and private Kaggle ROC-AUC. Class imbalance strategies produced different trade offs between false positives and false negatives.

The results show that no single configuration was best across all metrics and evaluation settings. Effective fraud detection requires consideration of feature representation, model choice, class imbalance handling, and evaluation objective.

Contents

Summary	4
Abstract	5
1 Introduction	10
1.1 Problem Statement	10
1.2 Research Questions	11
2 Literature Review and Related Work	12
2.1 Machine Learning for Fraud Detection	12
2.2 Class Imbalance	13
2.3 Feature Representation and Reduction	13
2.4 Machine Learning Models	14
2.5 Temporal Evaluation	14
2.6 Evaluation Metrics	15
2.7 Research Gap	16
3 Methodology	17
3.1 Research Design	17
3.2 Proposed Experimental Framework	18
3.3 Dataset	18
3.3.1 IEEE-CIS Dataset	18
3.3.2 Exploratory Data Analysis	19
3.3.3 Data Preparation	21

3.4	Train–Test Split	22
3.5	Feature Engineering and Reduction	23
3.5.1	Feature Representation	23
3.5.2	Feature Reduction	23
3.6	Class Imbalance Handling	24
3.7	Machine Learning Models	24
3.8	Hyperparameter Optimization	25
3.9	Experimental Design	25
3.10	Evaluation	26
4	Results	27
4.1	Evaluation	27
4.1.1	Best Performing Configurations	28
4.2	Effect of Feature Engineering	29
4.3	Effect of Feature Reduction	29
4.4	Multiple Metric Model Comparison	30
4.5	Error Analysis	32
4.6	Discussion	33
4.6.1	Feature Engineering	33
4.6.2	Feature Reduction	33
4.6.3	Model Performance Across Evaluation Settings	34
4.6.4	Effect of Class Imbalance Handling	35
4.6.5	Overall Implications	36
4.7	Limitations	37
4.8	Future Work	39
5	Conclusion	40

List of Figures

3.1	Proposed experimental framework for fraud detection evaluation.	18
3.2	Class distribution in the IEEE-CIS dataset	19
3.3	Correlation structure of the V features	20
3.4	Temporal distribution of IEEE-CIS transactions	21

List of Tables

4.1	Mean local and Kaggle ROC-AUC by model	27
4.2	Best performing configurations by evaluation setting	28
4.3	Aggregate effect of feature engineering	29
4.4	Aggregate effect of feature reduction	29
4.5	Mean local classification performance by model	31
4.6	Error profiles of XGBoost with Reduced Feature Engineering	32

Chapter 1

Introduction

1.1 Problem Statement

Financial fraud detection is a challenging machine learning problem because fraudulent transactions are rare, transaction data may contain hundreds of heterogeneous attributes, and transaction behavior can change over time. These characteristics can affect both model training and evaluation, particularly when models are developed using highly imbalanced and high dimensional data (Wei et al., 2013; Breskuvienė and Dzemyda, 2024; Ali et al., 2022; Dastidar et al., 2024).

Several methodological choices may influence fraud detection performance. Feature engineering can introduce temporal or entity related information that is not explicitly represented in the original attributes, while feature reduction can remove sparse, redundant, or weakly informative variables. Class imbalance handling such as oversampling and undersampling may alter trade off between detecting fraudulent transactions and incorrectly flagging legitimate transactions. Model selection further affects these outcomes because different learning algorithms respond differently to feature representation and class distribution.

Evaluation is another challenge for this topic. ROC-AUC alone may provide an incomplete view of performance under severe class imbalance, particularly when minority class prediction is of primary interest (Saito and Rehmsmeier, 2015; Dastidar et al., 2024). Fraud detection experiments benefit from temporal evaluation and complementary metrics that

capture both discrimination and minority class classification performance.

This study investigates these methodological factors using the IEEE-CIS Fraud Detection dataset, a large scale benchmark containing high dimensional transaction and identity data with severe class imbalance. Rather than proposing a new fraud detection algorithm, the study conducts a controlled empirical comparison of feature representation, feature reduction, class imbalance handling, and supervised machine learning models. Performance is evaluated on a chronologically held out test set and subsequently compared with the hidden IEEE-CIS Kaggle benchmark.

1.2 Research Questions

The study addresses the following research questions:

- **RQ1:** How does temporal and entity based feature engineering affect fraud detection performance compared with the original transaction features?
- **RQ2:** How does feature reduction affect dimensionality and fraud detection performance on the IEEE-CIS dataset?
- **RQ3:** Which supervised machine learning models perform best under severe class imbalance across different evaluation metrics and evaluation settings?
- **RQ4:** How do different class imbalance strategies affect fraud detection performance and classification errors?

Chapter 2

Literature Review and Related Work

2.1 Machine Learning for Fraud Detection

Financial fraud detection has evolved from rule based and statistical approaches toward machine learning methods that can identify complex patterns in transaction data (Ghosh and Reilly, 1994; Ali et al., 2022; Dastidar et al., 2024). Supervised learning is commonly used when historical transactions are labeled as legitimate or fraudulent. This allows classifiers to learn patterns from previous transactions and use them to predict unseen data.

Despite these developments, fraud detection remains challenging. Fraudulent transactions are usually rare compared with legitimate transactions, while fraud datasets may contain high dimensional and redundant attributes. Fraud patterns can also change over time (Ali et al., 2022; Dastidar et al., 2024). Model performance is therefore influenced by more than the choice of classifier. Feature representation, class imbalance handling, and evaluation design can also affect the results. Since previous studies use different combinations of these methods, their reported performance can be difficult to compare directly.

These challenges create a need for a controlled comparison of feature engineering, feature reduction, class imbalance handling, and model selection under consistent experimental conditions.

2.2 Class Imbalance

Class imbalance is a central challenge in fraud detection because fraudulent transactions generally represent only a small proportion of the available observations. A classifier can achieve high overall accuracy by correctly predicting the majority class while still detecting relatively few fraudulent transactions (Wei et al., 2013; Breskuvienė and Dzemyda, 2024).

Resampling techniques address this problem by changing the class distribution of the training data. The Synthetic Minority Over-sampling Technique (SMOTE) generates synthetic minority class observations from neighboring minority examples instead of simply duplicating existing samples (Chawla et al., 2002). Random undersampling uses a different approach by reducing the number of majority class observations. Both methods aim to improve minority class learning. However, they can produce different trade offs between detecting fraud and generating false positive predictions.

Previous research has reported benefits from resampling, although the results vary across classifiers, datasets, and evaluation metrics (Ali et al., 2022; Dastidar et al., 2024). A method that performs well under one experimental setting may not produce the same improvement under another. The original training distribution, SMOTE, and random undersampling are therefore compared under the same feature and model configurations in this study.

2.3 Feature Representation and Reduction

High dimensional fraud datasets may contain sparse, redundant, or weakly informative attributes. These attributes increase the size of the feature space and computational requirements without necessarily improving predictive performance. Feature selection and reduction methods address this problem by retaining useful information while removing unnecessary dimensions (Peng et al., 2005; Estévez et al., 2009; Vergara and Estévez, 2014).

Recent work on the IEEE-CIS dataset has combined missing value, correlation, and information based filtering with feature engineering to reduce the original feature space while maintaining predictive performance (Moradi et al., 2025). However, applying feature engineering and feature reduction together makes their individual effects difficult to distin-

guish. An improvement may come from newly constructed features, the removal of existing attributes, or the combination of both methods.

This study separates these factors by evaluating baseline and engineered feature representations both before and after feature reduction. The engineered representation introduces composite entity identifiers and temporal attributes derived from the available transaction information. As a result, feature engineering and feature reduction can be compared separately under the same experimental conditions.

2.4 Machine Learning Models

Fraud detection studies commonly compare several classifier families because model performance can vary with data representation, class distribution, and evaluation setting (Ali et al., 2022; Dastidar et al., 2024). Comparing different model families provides a broader view of their performance than evaluating variations of a single learning algorithm.

This study includes linear, single tree, bagging, and gradient boosting approaches. Logistic Regression provides a linear baseline, while Decision Tree models non linear decision boundaries through recursive feature partitioning. Random Forest combines randomized decision trees to reduce the instability associated with individual trees (Breiman, 2001). Three gradient boosting methods are also included. These are XGBoost (Chen and Guestrin, 2016), LightGBM (Ke et al., 2017), and CatBoost (Prokhorenkova et al., 2018).

The six algorithms represent different modeling characteristics and learning approaches. Using the same feature configurations, class imbalance handling methods, and temporal evaluation procedure provides a consistent basis for comparing their performance.

2.5 Temporal Evaluation

Evaluation design is particularly important in fraud detection because transaction behavior and fraud patterns can evolve over time. Randomly mixing earlier and later transactions across the training and test sets may produce performance estimates that differ from a setting where a model learns from historical transactions and then predicts future observations.

Non chronological evaluation has been identified as a limitation in fraud detection research, while temporally realistic validation more closely represents this prediction setting (Dastidar et al., 2024). A chronological split preserves the order of the transactions by training models on earlier observations and evaluating them on later observations.

A single chronological test set, however, represents only one evaluation setting. An additional benchmark provides another way to examine whether model performance and rankings change under a different evaluation setting. In this study, local evaluation on a chronologically held out test set is complemented by the hidden IEEE-CIS Kaggle benchmark. The two evaluations allow predictive performance and changes in model rankings to be compared.

2.6 Evaluation Metrics

Evaluation under severe class imbalance requires more than overall accuracy because strong performance on the majority class can conceal poor fraud detection. ROC-AUC measures ranking performance across classification thresholds and is also the official evaluation metric of the IEEE-CIS benchmark (Howard et al., 2019; Fawcett, 2006).

However, ROC-AUC does not provide a complete description of minority class performance. Precision–Recall analysis is particularly informative for imbalanced binary classification (Saito and Rehmsmeier, 2015). Precision measures the proportion of transactions predicted as fraudulent that are actually fraudulent. Recall measures the proportion of fraudulent transactions that are successfully detected. F1-score combines Precision and Recall to represent the balance between the two metrics.

Balanced Accuracy and Matthews Correlation Coefficient (MCC) provide additional views of classification performance when the class distribution is unequal. This study uses ROC-AUC, PR-AUC, Precision, Recall, F1-score, Balanced Accuracy, MCC, and confusion matrix values. Using these metrics together helps distinguish differences in fraud detection and false positive behavior even when models achieve similar ROC-AUC values.

2.7 Research Gap

Previous fraud detection research has investigated machine learning classifiers, feature engineering, feature selection, and methods for handling class imbalance. These studies provide evidence that each choice can affect fraud detection performance, but their reported results are difficult to compare directly when preprocessing, sampling, model configuration, and evaluation procedures differ across experiments.

This is particularly important for imbalanced fraud data. Improvements in ROC-AUC do not necessarily correspond to improvements in Precision, Recall, or false positive behavior, while resampling methods can favor different classification objectives. Evaluation design introduces a further source of variation because results obtained from one data split may not represent model behavior on later transactions.

A remaining need is therefore to examine these methodological choices under a common experimental framework rather than evaluating their reported performance across separate studies. This study addresses that need by comparing feature representation, feature reduction, class imbalance handling, and six supervised machine learning models under the same chronological evaluation procedure. Multiple classification metrics and confusion matrix errors are used alongside the hidden IEEE-CIS Kaggle benchmark to examine whether conclusions remain consistent across metrics and evaluation settings.

Chapter 3

Methodology

3.1 Research Design

This study adopts a quantitative experimental design to examine how feature representation, feature reduction, class imbalance handling, and model selection affect fraud detection performance. These factors address common challenges in machine learning based fraud detection, including high dimensional data, severe class imbalance, and changes in transaction behavior over time (Ali et al., 2022; Dastidar et al., 2024).

The experimental workflow includes data preparation, feature representation and reduction, chronological data partitioning, class imbalance handling, model training and optimization, local temporal evaluation, and Kaggle benchmark evaluation. A consistent experimental procedure is maintained across these stages to support comparison between different configurations.

A total of 72 experimental configurations are evaluated. These configurations combine four feature configurations, three class imbalance handling conditions, and six supervised machine learning models. All configurations follow the same chronological evaluation procedure so that their performance can be compared under consistent conditions.

3.2 Proposed Experimental Framework

Figure 3.1 presents the proposed experimental framework used in this study. The framework organizes the experiment around feature representation, feature reduction, class imbalance handling, model selection, and evaluation. It also preserves the temporal order of transactions by separating earlier transactions used for model development from later transactions used for final evaluation.

The same framework is applied across the 72 experimental configurations. This provides a common procedure for comparing the effects of different feature configurations, class imbalance handling methods, and machine learning models before their performance is examined using the local chronological test set and the hidden Kaggle benchmark.

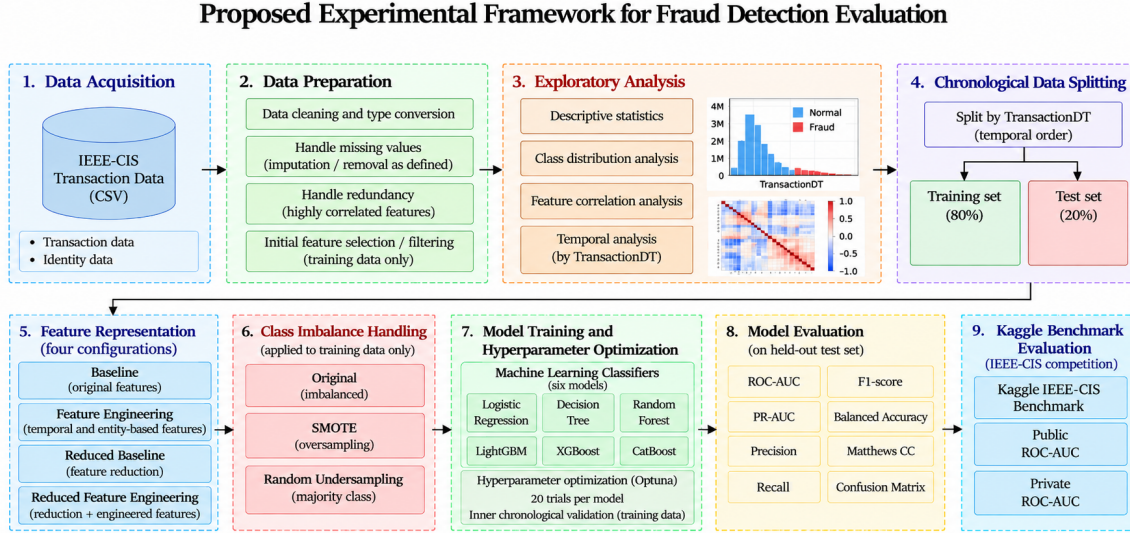


Figure 3.1: Proposed experimental framework for fraud detection evaluation.

3.3 Dataset

3.3.1 IEEE-CIS Dataset

The empirical evaluation uses the IEEE-CIS Fraud Detection dataset, released through Kaggle with transaction data provided by Vesta Corporation (Howard et al., 2019). The labeled training data contain 590,540 transactions, of which approximately 3.5% are fraudulent.

Each transaction is identified by `TransactionID`, while the binary target `isFraud` identifies whether the transaction is fraudulent.

The dataset is provided in separate transaction and identity tables. The transaction table contains payment, card, address, email domain, temporal, and anonymized attributes. The identity table provides additional identity and device information for a subset of transactions (Howard et al., 2019). These tables are merged using `TransactionID` to construct the dataset used for modeling.

3.3.2 Exploratory Data Analysis

Exploratory data analysis was conducted before model development to examine characteristics relevant to preprocessing and evaluation. Since many IEEE-CIS attributes are anonymized, the analysis focuses on their statistical properties without assigning undocumented business meanings to these features.

Class Imbalance and Missingness

The labeled data are strongly imbalanced, with fraudulent transactions representing approximately 3.5% of the observations. Figure 3.2 shows the distribution between legitimate and fraudulent transactions.

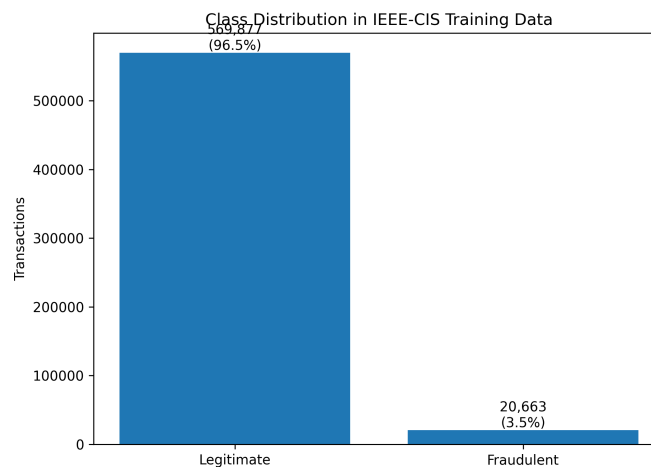


Figure 3.2: Distribution of legitimate and fraudulent transactions in the labeled IEEE-CIS data.

The proportion of missing data also varies considerably across the feature space. Identity attributes contain particularly high levels of missing data because identity records are available for only a subset of transactions. Several transaction attributes also contain substantial missing values. These observations provide a basis for the missing value filtering applied during feature reduction.

Feature Redundancy

Correlation analysis shows substantial redundancy among numerical attributes, particularly within the anonymized V1-V339 feature group. Figure 3.3 compares the correlation structure of the complete V feature group with a representative reduced subset.

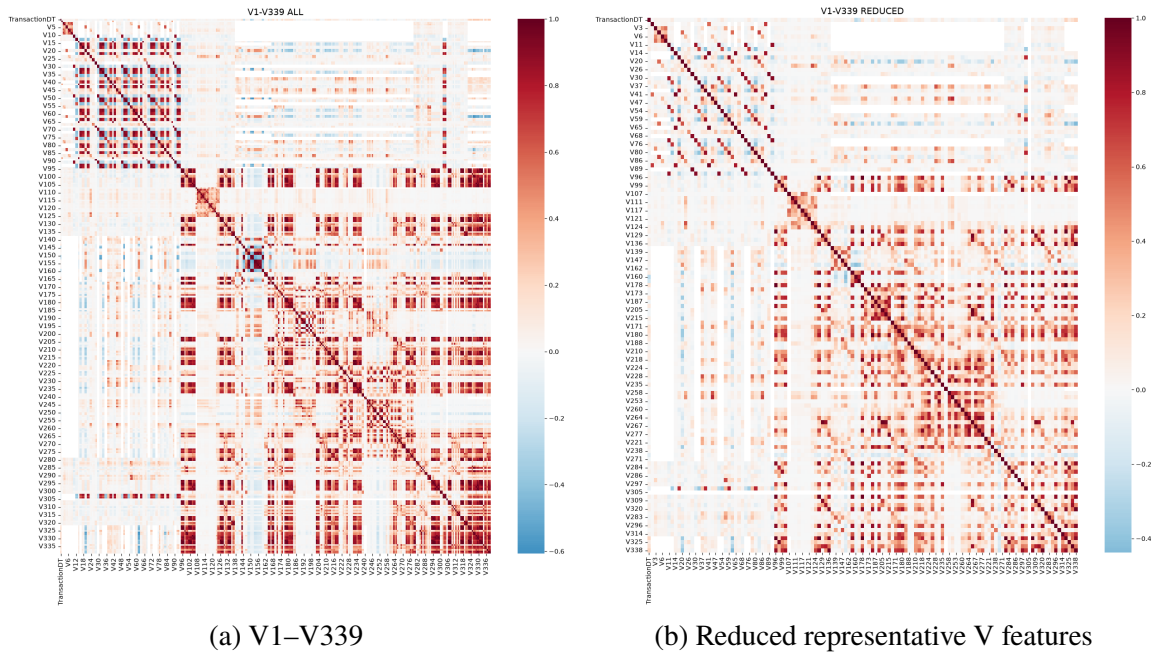


Figure 3.3: Correlation structure of the anonymized V feature group before and after exploratory reduction.

Correlation is also present within other numerical feature groups. Since the meanings of many attributes are undisclosed, these relationships are treated as statistical evidence of potential redundancy. This observation supports the use of correlation based feature reduction in the experiments.

Temporal Structure

`TransactionDT` represents elapsed time from an undisclosed reference rather than an absolute calendar date. However, it preserves the temporal ordering of transactions. Figure 3.4 shows that the labeled training data and unlabeled competition data occupy successive temporal regions.

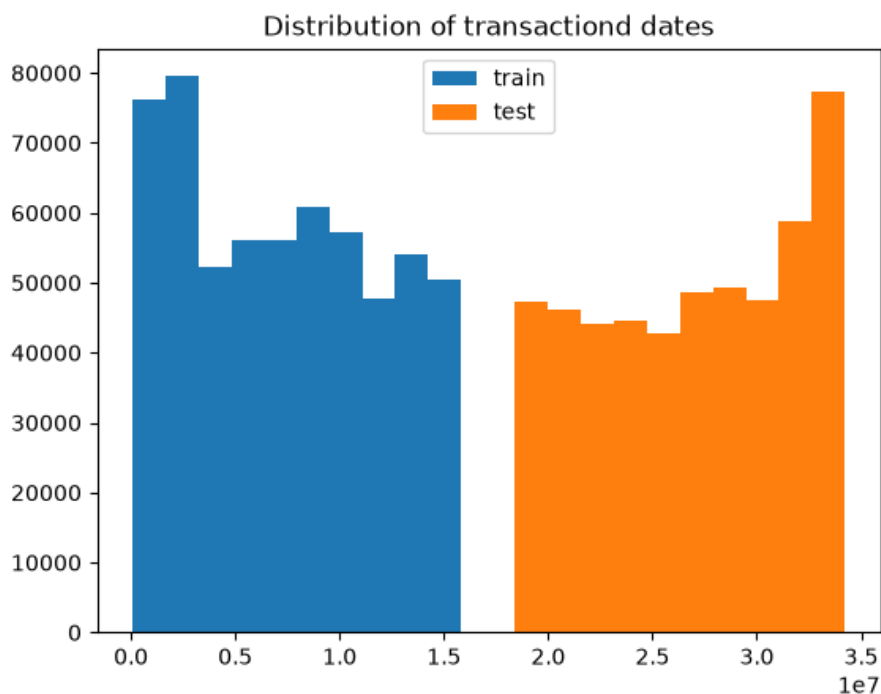


Figure 3.4: Distribution of `TransactionDT` in the labeled training data and unlabeled competition data.

The temporal structure supports chronological evaluation rather than random partitioning. Together, the exploratory analysis identifies three characteristics that are important to the experimental design, including class imbalance, feature redundancy, and temporal ordering.

3.3.3 Data Preparation

The transaction and identity tables are merged using a left join on `TransactionID`. This preserves all transaction records even when corresponding identity information is unavailable. `TransactionID` is retained for record identification and Kaggle prediction submission but

is excluded from the predictive feature set.

Categorical attributes are converted to numerical representations using label encoding. For preprocessing of the IEEE-CIS data, the categorical mapping is constructed from the labeled dataset and then applied to the unlabeled competition data. Categories in the competition data that are not observed in the labeled data are assigned a separate value. Binary categorical attributes represented by T and F are mapped to numerical values, with missing values represented separately.

Missing numerical values are replaced with -999 before model training. This provides a consistent numerical representation of missing observations across the experimental configurations.

Separate Baseline and Feature Engineering datasets are created during preprocessing. Feature reduction is then applied to these representations to produce the four experimental feature configurations.

3.4 Train–Test Split

Transactions are ordered by `TransactionDT` and divided chronologically rather than randomly. The earliest 80% of transactions form the training set, while the latest 20% form the local test set. This partition preserves the temporal order of the data and better represents a setting in which earlier transactions are used to predict later transactions (Dastidar et al., 2024).

For hyperparameter optimization, the training set is further divided chronologically. The earliest 80% of this training set is used for inner model fitting, while the latest 20% is used for validation. The final test set remains separate from feature selection, hyperparameter optimization, and model selection.

After optimization, the selected model configuration is retrained on the complete training set and evaluated on the chronologically later test set.

3.5 Feature Engineering and Reduction

3.5.1 Feature Representation

Two feature representations are constructed. The *Baseline* representation contains the original predictive IEEE-CIS attributes, excluding the target and record identifier. The *Feature Engineering* representation extends this feature set with two composite identifiers, `uid` and `uid2`, together with `DT_month`, `DT_week`, `DT_day`, `DT_weekday`, and `DT_hour`.

The composite identifiers combine available card, address, and email domain attributes to represent recurring combinations of transaction characteristics. The temporal features are derived from `TransactionDT` using a fixed reference date. Since the original reference date is undisclosed, these values represent relative transaction time rather than actual transaction dates.

3.5.2 Feature Reduction

Feature reduction applies three sequential criteria to remove highly sparse, redundant, and weakly informative predictors. First, features with at least 90% missing values are removed.

The remaining numerical features are then examined using absolute Pearson correlation. When the absolute correlation between two features exceeds 0.98, the later feature in the feature ordering is removed while the earlier feature is retained. Missing values are temporarily treated as missing and replaced by the training median for the correlation calculation.

The final stage uses mutual information between each remaining feature and the fraud target. Mutual information is estimated using `mutual_info_classif` with three nearest neighbors and a maximum stratified sample of 80,000 observations. Features with a mutual information value below 0.001 are removed.

Feature reduction is determined using only the chronological training data. The final test set does not contribute to feature selection.

After reduction, the four experimental representations contain 432 Baseline features, 439 Feature Engineering features, 342 Reduced Baseline features, and 348 Reduced Feature Engineering features.

3.6 Class Imbalance Handling

Three training distribution conditions are evaluated. These are the original class distribution, SMOTE, and random undersampling. SMOTE generates synthetic minority observations from neighboring minority examples (Chawla et al., 2002), while random undersampling reduces the number of majority class observations.

Both resampling procedures produce a 1:1 class distribution and are applied only to the training data. SMOTE uses five nearest neighbors, and both resampling procedures use a random seed of 42. Validation and test data retain their original class distributions.

Models that support class weighting calculate their weights from the training distribution supplied to the model. Under the original imbalanced distribution, this assigns greater weight to fraudulent transactions. After SMOTE or random undersampling produces a balanced 1:1 training distribution, the resulting class weights become approximately equal. XGBoost similarly calculates `scale_pos_weight` from the corresponding training labels, resulting in a value close to 1 after resampling.

The three conditions therefore compare model behavior under the original imbalanced training distribution with behavior after SMOTE or random undersampling has changed the training distribution. SMOTE is applied after categorical attributes have been numerically encoded. Standard SMOTE therefore operates on the encoded feature space without distinguishing categorical attributes. The limitation of this approach is discussed in Section 4.7.

3.7 Machine Learning Models

Six supervised classifiers are evaluated, including Logistic Regression, Decision Tree, Random Forest, LightGBM, XGBoost, and CatBoost. Logistic Regression applies standard feature scaling and balanced class weights. Decision Tree, Random Forest, and LightGBM also use balanced class weights. XGBoost uses `scale_pos_weight`, calculated from the ratio of legitimate to fraudulent transactions in the corresponding training data, while CatBoost uses automatic balanced class weights.

The same six model families are evaluated across all feature and sampling conditions.

This provides a consistent basis for comparing their performance across the experimental configurations.

3.8 Hyperparameter Optimization

Hyperparameters are optimized using Optuna with a Tree structured Parzen Estimator sampler and a fixed seed of 42. Each model and feature configuration is optimized for 20 trials using ROC-AUC on the chronological inner validation set as the objective.

The configuration with the highest validation ROC-AUC is then retrained using the complete training set and evaluated on the final chronological test set. For Random Forest, optimization uses the most recent 100,000 observations from the inner training set to reduce computational cost, while final model fitting still uses the complete training set.

The selected hyperparameters are reused for the corresponding SMOTE and random undersampling experiments rather than being optimized again after resampling. This keeps the model parameters consistent when comparing the effects of the three training distribution conditions.

3.9 Experimental Design

The primary experiment combines four feature configurations, three class imbalance handling conditions, and six supervised machine learning models. This produces a total of 72 experimental configurations.

All configurations are evaluated using the same chronological test set. For each model and feature combination, the selected hyperparameters are also reused across the corresponding resampling experiments. This allows the effect of class imbalance handling to be compared without adding a separate optimization stage for each sampling condition.

3.10 Evaluation

Local evaluation is performed on the chronologically held out 20% test set. The recorded metrics include Accuracy, Precision, Recall, F1-score, ROC-AUC, PR-AUC, Balanced Accuracy, and MCC. The TN, FP, FN, and TP values from the confusion matrix are also recorded to examine classification errors.

For threshold dependent metrics, including Precision, Recall, F1-score, Balanced Accuracy, MCC, and the confusion matrix, class predictions are obtained using the default binary classification threshold of 0.5. ROC-AUC and PR-AUC are calculated from predicted fraud probabilities and then evaluate ranking performance independently of this fixed classification threshold.

Kaggle benchmark evaluation uses the hidden IEEE-CIS competition data (Howard et al., 2019). Each of the 72 configurations is fitted using the available labeled training data and used to generate fraud probabilities for the hidden competition data. The corresponding public and private Kaggle ROC-AUC scores are recorded and compared with the local results.

The Kaggle scores are not used during hyperparameter optimization. Instead, they provide an additional evaluation of model performance on the hidden competition data and allow differences between the local chronological test set and the Kaggle benchmark to be examined. The Kaggle scores are used descriptively to compare the evaluated configurations rather than to select hyperparameters. Consequently, the configuration with the highest Kaggle score is reported as a benchmark result rather than as an independently selected final model.

Chapter 4

Results

4.1 Evaluation

The 72 experimental configurations were evaluated on the chronologically held out test set and then compared with the hidden IEEE-CIS Kaggle benchmark. ROC-AUC provides a common measure for this comparison because both the local evaluation and Kaggle benchmark report this metric.

Table 4.1 presents the mean ROC-AUC across the 12 feature and sampling configurations evaluated for each model. The generalization gap is calculated as

$$\text{Gap} = \text{Local ROC-AUC} - \text{Kaggle Private ROC-AUC},$$

where a positive value indicates that the private Kaggle ROC-AUC is lower than the local ROC-AUC, while a negative value indicates that the private Kaggle ROC-AUC is higher.

Table 4.1: Mean local and Kaggle ROC-AUC by model

Model	Local	Public	Private	Gap
Logistic Regression	0.8278	0.8551	0.7920	0.0358
Decision Tree	0.8136	0.8977	0.8736	-0.0601
Random Forest	0.8862	0.9222	0.8992	-0.0130
LightGBM	0.9163	0.8856	0.8378	0.0785
XGBoost	0.9161	0.9207	0.8909	0.0252
CatBoost	0.9043	0.9290	0.9010	0.0032

LightGBM recorded the highest mean local ROC-AUC at 0.9163, with XGBoost effectively similar at 0.9161. Given the absence of repeated temporal evaluations or confidence intervals, this small difference should not be interpreted as evidence that LightGBM consistently outperforms XGBoost. The Kaggle benchmark showed a different pattern. CatBoost produced the highest mean public and private Kaggle scores at 0.9290 and 0.9010, respectively. It also had the smallest absolute difference between local and private ROC-AUC at 0.0032.

LightGBM showed the largest positive gap. Its mean ROC-AUC decreased from 0.9163 locally to 0.8378 on the private leaderboard. Decision Tree and Random Forest showed the opposite pattern, with higher mean private ROC-AUC than local ROC-AUC. These differences show that model rankings changed between the chronological local evaluation and the hidden Kaggle benchmark.

4.1.1 Best Performing Configurations

Table 4.2 compares the configurations with the highest ROC-AUC in each evaluation setting.

Table 4.2: Best performing configurations by evaluation setting

Best by	Model	Sampling	Features	Local	Public	Private
Local	XGBoost	SMOTE	Reduced FE	0.9217	0.9294	0.8981
Public	Decision Tree	SMOTE	Reduced FE	0.8136	0.9367	0.8997
Private	Decision Tree	Undersampling	Reduced FE	0.8347	0.9336	0.9128

The best local result was produced by XGBoost with SMOTE and Reduced Feature Engineering, reaching a ROC-AUC of 0.9217. On the public leaderboard, Decision Tree with SMOTE and Reduced Feature Engineering reached the highest score of 0.9367. The private leaderboard favored Decision Tree with undersampling and Reduced Feature Engineering, which reached 0.9128.

Reduced Feature Engineering was common to all three configurations. However, the preferred model and class imbalance handling method changed across evaluation settings. The following sections examine the experimental factors across all configurations rather than drawing conclusions from individual best performing results.

4.2 Effect of Feature Engineering

The aggregate effect of feature engineering was examined by comparing the Baseline and Reduced Baseline configurations with their corresponding Feature Engineering configurations across all models and class imbalance handling conditions.

Table 4.3: Aggregate effect of feature engineering

Representation	ROC-AUC	PR-AUC	Precision	Recall	Private
Without engineered features	0.8775	0.4279	0.3316	0.6203	0.8648
With engineered features	0.8772	0.4288	0.3360	0.6203	0.8667

Feature engineering produced little change in aggregate performance. Mean local ROC-AUC changed slightly from 0.8775 to 0.8772, while Recall remained unchanged at 0.6203. PR-AUC and Precision increased from 0.4279 to 0.4288 and from 0.3316 to 0.3360, respectively. A small increase was also observed in private ROC-AUC, from 0.8648 to 0.8667.

Overall, the addition of `uid`, `uid2`, and temporal features did not consistently improve performance across all models and class imbalance handling conditions. Their effect varied with the surrounding model and feature configuration.

4.3 Effect of Feature Reduction

Feature reduction decreased the Baseline representation from 432 to 342 features and the Feature Engineering representation from 439 to 348 features. In both cases, approximately 21% of the predictor space was removed.

Table 4.4: Aggregate effect of feature reduction

Representation	ROC-AUC	PR-AUC	Precision	Recall	Private
Unreduced	0.8764	0.4300	0.3289	0.6243	0.8583
Reduced	0.8783	0.4267	0.3388	0.6163	0.8732

Mean local ROC-AUC was similar before and after feature reduction, changing from 0.8764 to 0.8783. Precision increased from 0.3289 to 0.3388, while PR-AUC and Recall decreased from 0.4300 to 0.4267 and from 0.6243 to 0.6163. A larger difference appeared on the private Kaggle leaderboard, where mean ROC-AUC increased from 0.8583 to 0.8732.

Feature reduction removed approximately one fifth of the predictors while maintaining similar aggregate local ROC-AUC. The reduced representations also produced higher mean private Kaggle ROC-AUC, although the changes in PR-AUC and Recall show that the effect differed across evaluation metrics.

The controlled XGBoost error analysis makes these differences clearer. With Reduced Feature Engineering, SMOTE reduced false positives from 1,593 under the original distribution to 391, while false negatives increased from 1,899 to 2,513. Undersampling produced the opposite pattern. False negatives decreased to 990, while false positives increased to 11,315. Compared with SMOTE, undersampling detected 1,523 additional fraudulent transactions but generated 10,924 additional false positive predictions.

These differences should also be considered in relation to the fixed classification threshold. Precision, Recall, F1-score, Balanced Accuracy, and the confusion matrix are calculated from class predictions at a threshold of 0.5. Resampling changes the training distribution and can also change the distribution of predicted fraud probabilities. Part of the observed Precision and Recall trade off may therefore reflect changes in classification behavior at this threshold.

This interpretation is supported by the ranking metrics. The original training distribution achieved the highest mean PR-AUC at 0.4421, while its mean ROC-AUC of 0.8797 was similar to the 0.8807 obtained with undersampling. SMOTE produced a lower mean ROC-AUC of 0.8717. The substantial changes in Precision and Recall were therefore not accompanied by a consistent improvement in ranking performance.

The preferred training distribution depends on the evaluation objective. Undersampling produced greater fraud sensitivity at the default threshold, while SMOTE produced higher Precision, F1, and MCC. Threshold selection and probability calibration could change these trade offs and are not investigated in the present study.

4.4 Multiple Metric Model Comparison

Table 4.5 summarizes local model performance averaged across the feature and class imbalance handling configurations.

Table 4.5: Mean local classification performance by model

Model	ROC-AUC	PR-AUC	Precision	Recall	F1	Bal. Acc.	MCC
Logistic Regression	0.8278	0.1900	0.0882	0.7934	0.1587	0.7502	0.1975
Decision Tree	0.8136	0.3011	0.1402	0.6069	0.2188	0.7289	0.2334
Random Forest	0.8862	0.4683	0.3916	0.5523	0.3634	0.7413	0.3829
LightGBM	0.9163	0.5478	0.4583	0.5943	0.4391	0.7737	0.4569
XGBoost	0.9161	0.5541	0.5234	0.5627	0.4680	0.7618	0.4823
CatBoost	0.9043	0.5088	0.4011	0.6122	0.3784	0.7723	0.4097

LightGBM recorded the highest mean local ROC-AUC at 0.9163 and the highest Balanced Accuracy at 0.7737. XGBoost was close in ROC-AUC at 0.9161 and performed better across several other metrics, including PR-AUC, Precision, F1, and MCC. Among the models evaluated locally, XGBoost therefore provided the strongest performance across the broader set of minority class metrics.

Logistic Regression produced the highest Recall at 0.7934, but its Precision was only 0.0882. This combination shows that detecting a larger proportion of fraudulent transactions can also result in a high number of legitimate transactions being classified as fraud. CatBoost did not lead the local metrics. Its stronger performance appeared on the Kaggle benchmark, where it produced the highest mean public and private ROC-AUC and the smallest absolute local to private difference, as reported in Table 4.1.

The model comparison does not identify a single model that performs best across every metric and evaluation setting. LightGBM and XGBoost produced the strongest local results, while CatBoost achieved the highest mean public and private Kaggle ROC-AUC.

4.5 Error Analysis

To examine how class imbalance handling changes classification errors under a controlled setting, Table 4.6 compares XGBoost with Reduced Feature Engineering across the three sampling conditions.

Table 4.6: Error profiles of XGBoost with Reduced Feature Engineering

Sampling	TN	FP	FN	TP	Precision	Recall	F1	MCC
Original	112451	1593	1899	2165	0.5761	0.5327	0.5536	0.5387
SMOTE	113653	391	2513	1551	0.7987	0.3816	0.5165	0.5421
Undersampling	102729	11315	990	3074	0.2136	0.7564	0.3332	0.3662

SMOTE produced a more conservative error profile. False positives decreased from 1,593 under the original condition to 391, while false negatives increased from 1,899 to 2,513. Undersampling showed the opposite pattern. False negatives decreased to 990, but false positives increased substantially to 11,315.

Compared with SMOTE, undersampling detected 1,523 additional fraudulent transactions on the same test set. However, this improvement came with 10,924 additional false positive predictions. The original condition remained between these two error profiles.

The confusion matrix results provide a clearer view of the trade off between Precision and Recall. Undersampling favors the detection of more fraudulent transactions, although this also increases the number of legitimate transactions incorrectly classified as fraud. SMOTE reduces this false positive burden, but at the cost of missing more fraudulent transactions. The preferred strategy therefore depends on whether the application places greater importance on fraud detection or on limiting unnecessary alerts.

The primary experiment retains aggregate confusion matrix counts for all configurations. This analysis can therefore compare the frequency of classification errors, but it does not identify whether these errors are concentrated within particular types of transactions.

4.6 Discussion

4.6.1 Feature Engineering

The results provide limited evidence that the engineered entity and temporal features consistently improve fraud detection performance. Across all models and sampling conditions, adding the engineered features changed mean local ROC-AUC only marginally, from 0.8775 to 0.8772. PR-AUC, Precision, and private Kaggle ROC-AUC improved slightly, while Recall was effectively unchanged. Feature engineering therefore affected some classification characteristics without producing a clear overall improvement.

One explanation is that the original IEEE-CIS attributes already contain substantial information related to transaction identity and time. The engineered uid and uid2 features combine existing card, address, and email-domain attributes, while the temporal features are derived directly from TransactionDT. Their information may therefore overlap with patterns already recoverable from the original feature space, particularly for tree based ensemble models.

The aggregate result does not imply that the engineered features are universally uninformative. The highest local, public, and private ROC-AUC configurations all used Reduced Feature Engineering. This indicates that engineered features can contribute to particular model and sampling combinations even though their average effect across all 72 configurations is small. Feature engineering should be evaluated jointly with the surrounding modeling pipeline rather than assumed to improve performance independently.

4.6.2 Feature Reduction

Feature reduction produced a more consistent benefit than feature engineering. The reduction procedure decreased the Baseline feature space from 432 to 342 predictors and the Feature Engineering space from 439 to 348 predictors, removing approximately one fifth of the available features. Despite this reduction, mean local ROC-AUC increased slightly from 0.8764 to 0.8783.

A larger difference appeared on the private Kaggle leaderboard. Mean private ROC-

AUC increased from 0.8583 for unreduced configurations to 0.8732 after reduction. The reduced feature representations therefore performed better on this hidden benchmark evaluation, although the present experiment does not isolate which part of the reduction procedure is responsible for the difference.

The improvement was not uniform across all metrics. Feature reduction slightly decreased mean PR-AUC and Recall while increasing Precision and Kaggle ROC-AUC. It should therefore not be interpreted as universally improving classification performance. Rather, the results show that substantial dimensionality reduction was possible without sacrificing aggregate ROC-AUC and with improved performance on the private Kaggle benchmark.

This distinction is important for fraud detection. Within the IEEE-CIS benchmark, retaining every available attribute did not consistently produce better predictive performance. Feature reduction provided a smaller representation while maintaining similar mean local ROC-AUC, although its effects differed across the other evaluation metrics.

4.6.3 Model Performance Across Evaluation Settings

The model comparison demonstrates that the choice of “best” model depends on both the evaluation metric and the evaluation distribution. LightGBM achieved the highest mean local ROC-AUC, while XGBoost achieved the strongest mean PR-AUC, Precision, F1, and MCC. These results indicate that the two boosting models provided the strongest overall local performance, but they emphasized different aspects of classification quality.

The Kaggle benchmark produced a different model ranking. CatBoost achieved the highest mean public and private Kaggle ROC-AUC and showed only a small difference between mean local and private performance. In contrast, LightGBM achieved the highest mean local ROC-AUC but the largest positive local to private gap. The model with the highest mean local ROC-AUC was therefore not the model with the highest mean Kaggle ROC-AUC.

The individual winning configurations reinforce this result. XGBoost with SMOTE and Reduced Feature Engineering achieved the highest local ROC-AUC, whereas Decision Tree

configurations achieved the highest public and private leaderboard scores. These outcomes show that selecting a model from a single local metric can produce a different choice from selecting it using the Kaggle leaderboard results.

The comparison also illustrates why model evaluation should not rely on ROC-AUC alone. Logistic Regression achieved the highest mean Recall but very low Precision, while XGBoost achieved a stronger balance across PR-AUC, Precision, F1-score, and MCC. CatBoost achieved the highest mean public and private Kaggle ROC-AUC despite not leading most local classification metrics. Model selection for imbalanced fraud detection should therefore consider minority class performance and results across multiple evaluation settings in addition to ROC-AUC.

4.6.4 Effect of Class Imbalance Handling

The sampling experiments demonstrate that class imbalance handling changes the type of classification errors rather than producing a single universally superior strategy. SMOTE achieved the highest mean Precision, F1, MCC, and private Kaggle ROC-AUC, whereas undersampling achieved the highest Recall and Balanced Accuracy. The original training distribution produced the highest mean PR-AUC.

The controlled XGBoost error analysis makes this trade off more concrete. With Reduced Feature Engineering, SMOTE reduced false positives from 1,593 under the original distribution to 391, but increased false negatives from 1,899 to 2,513. Undersampling produced the opposite effect, reducing false negatives to 990 while increasing false positives to 11,315. Relative to SMOTE, undersampling therefore identified 1,523 additional fraudulent transactions at the cost of 10,924 additional false-positive predictions.

Neither outcome can be considered universally preferable without considering the relative cost of the two error types. In an environment where missing fraud has a high cost, greater Recall may justify a larger false-positive workload. Conversely, where manual review capacity or customer disruption is a major constraint, higher Precision may be preferred.

The interpretation of these results also depends on how model level class weighting in-

interacts with resampling. Under the original imbalanced distribution, models that support automatic class weighting assign greater weight to fraudulent transactions. After SMOTE or random undersampling produces a 1:1 training distribution, these weights become approximately equal. XGBoost similarly obtains a `scale_pos_weight` close to 1. The observed differences therefore reflect the complete training configurations produced by each class imbalance handling condition rather than the isolated effect of resampling added to unchanged class weights.

4.6.5 Overall Implications

The experiments show that fraud detection performance is influenced by interactions among feature representation, class imbalance handling, model choice, evaluation metric, and evaluation setting rather than by a single component of the pipeline.

Feature engineering alone produced little average improvement, whereas feature reduction removed approximately one fifth of the predictor space while maintaining similar mean local ROC-AUC and producing higher mean private Kaggle ROC-AUC. Boosting models performed strongly under the local evaluation, but their relative rankings changed on the Kaggle benchmark. Similarly, class imbalance handling changed the balance between false positives and false negatives more substantially than it changed ROC-AUC.

These findings support evaluating fraud detection pipelines as complete experimental configurations rather than considering individual components in isolation. Improvements observed using one metric or one evaluation setting should not automatically be interpreted as improvements in minority class classification or performance under another evaluation setting. Multiple metrics and temporally ordered evaluation provide a broader basis for comparing the evaluated fraud detection configurations.

4.7 Limitations

Several limitations should be considered when interpreting the results of this study.

Firstly, the empirical evaluation uses only the IEEE-CIS Fraud Detection dataset. Many attributes are anonymized and their underlying meanings are undisclosed, which limits interpretation of individual predictors. The findings may also differ across other fraud datasets or transaction systems.

Secondly, local evaluation uses a single chronological hold out split. Although this preserves temporal ordering, performance may depend on the selected temporal boundary. The Kaggle benchmark provides an additional evaluation on hidden data from the same competition, but does not establish generalization to independent fraud datasets. The study also reports point estimates rather than confidence intervals from repeated temporal partitions or training runs. Small differences between configurations should therefore be interpreted cautiously.

Categorical mappings were constructed from the full labeled dataset before the chronological model partition was applied. Although this procedure does not use the fraud target, category information from observations later assigned to the local test set was therefore available during preprocessing. Future evaluation should fit such preprocessing mappings using only the development data and apply them unchanged to later observations.

Thirdly, hyperparameter optimization was limited to 20 Optuna trials per model and feature configuration. Random Forest optimization used the most recent 100,000 observations from the inner training data, and the selected hyperparameters were reused for the corresponding SMOTE and undersampling experiments. Larger optimization budgets or separate optimization for each sampling condition could produce different results.

The evaluation uses statistical classification metrics without assigning financial or operational costs to false positives and false negatives. Classification thresholds were also not optimized for a specific deployment objective. In addition, transaction level predictions were not retained across all configurations, limiting the error analysis to aggregate confusion matrix behavior.

Finally, SMOTE was applied after categorical attributes had been numerically encoded.

Because standard SMOTE interpolates observations in numerical feature space without distinguishing categorical attributes, some synthetic values may not correspond directly to valid categorical states. Methods designed for mixed numerical and categorical data could be investigated in future work.

4.8 Future Work

Future work could extend the evaluation to additional fraud datasets and time periods to examine whether the observed results remain consistent across different transaction environments. Rolling or walk forward evaluation could also provide a more detailed assessment of model performance as transaction patterns change over time.

The optimization procedure could be expanded using larger hyperparameter search budgets and separate optimization for each class imbalance strategy. Alternative methods for mixed numerical and categorical data could also be investigated instead of applying standard SMOTE to numerically encoded categorical attributes.

Future experiments could incorporate the operational costs of false positives and false negatives when selecting classification thresholds. Retaining transaction level predictions would also enable more detailed error analysis to identify transaction characteristics associated with false positives and missed fraud.

Finally, graph based approaches could be investigated by representing transactions and related entities as connected structures. Graph neural networks may provide a useful direction for examining relationships between transactions, payment attributes, and other shared entities that are not explicitly represented by the supervised models evaluated in this study.

Chapter 5

Conclusion

This study investigated how feature representation, feature reduction, model selection, and class imbalance handling affect machine learning based fraud detection on the IEEE-CIS Fraud Detection dataset. Six supervised learning models were evaluated across 72 experimental configurations using a chronological hold out test set and the hidden Kaggle benchmark. Performance was assessed using multiple metrics to capture both ranking performance and minority class classification.

For RQ1, the engineered entity and temporal features produced limited improvement on average compared with the original transaction features. The strongest individual configurations included the engineered features, but the aggregate results indicate that feature engineering alone did not consistently improve fraud detection performance.

For RQ2, feature reduction removed approximately one fifth of the predictors while producing similar mean local ROC-AUC. Reduced configurations achieved higher mean private Kaggle ROC-AUC, although small decreases were observed in PR-AUC and Recall. Within the evaluation used in this study, feature reduction produced a smaller feature space without a substantial change in mean local ROC-AUC.

For RQ3, no single supervised model performed best across all metrics and evaluation settings. LightGBM achieved the highest mean local ROC-AUC and Balanced Accuracy, while XGBoost performed best across several minority class metrics, including PR-AUC, Precision, F1-score, and MCC. CatBoost achieved the highest mean public and private Kaggle ROC-AUC. These differences show that model selection depends on the evaluation ob-

jective rather than a single performance measure.

For RQ4, the class imbalance strategies produced different classification trade offs. SMOTE configurations achieved higher mean Precision, F1-score, and MCC, while random undersampling achieved the highest mean Recall and Balanced Accuracy. The original training distribution achieved the highest mean PR-AUC. Error analysis further showed that undersampling detected more fraudulent transactions at the cost of substantially more false positives, while SMOTE reduced false positives but missed more fraud. The preferred strategy therefore depends on the relative importance of these error types.

The principal contribution of this study is a controlled empirical comparison of these methodological choices within the IEEE-CIS Fraud Detection benchmark under a consistent temporal evaluation framework. By evaluating complete fraud detection configurations using multiple metrics and hidden Kaggle test data, the study provides evidence of how feature representation, feature reduction, model selection, and class imbalance handling interact within this benchmark. The results reinforce the importance of evaluating fraud detection systems using multiple criteria and evaluation settings rather than selecting a model from a single metric.

Bibliography

- Abdulalem Ali, Shukor Abd Razak, Siti Hajar Othman, Taiseer Abdalla Elfadil Eisa, Arafat Al-Dhaqm, Maged Nasser, Tusneem Elhassan, Hashim Elshafie, and Abdu Saif. Financial fraud detection based on machine learning: A systematic literature review. *Applied Sciences*, 12(19), 2022. ISSN 2076-3417. doi: 10.3390/app12199637. URL <https://www.mdpi.com/2076-3417/12/19/9637>.
- Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001. doi: 10.1023/A:1010933404324.
- Dalia Breskuvienė and Gintautas Dzemyda. Enhancing credit card fraud detection: highly imbalanced data case. *Journal of Big Data*, 11(1): 182, 2024. ISSN 2196-1115. doi: 10.1186/s40537-024-01059-5. URL <https://doi.org/10.1186/s40537-024-01059-5>.
- Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.
- Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM, 2016. doi: 10.1145/2939672.2939785.
- K. G. Dastidar, O. Caelen, and M. Granitzer. Machine learning methods for credit card fraud detection: A survey. *IEEE Access*, 12:158939–158965, 2024. doi: 10.1109/ACCESS.2024.3487298.
- Pablo A. Estévez, Marco Tesmer, Claudio A. Perez, and Jacek M. Zurada. Normalized mutual information feature selection. *IEEE Transactions on Neural Networks*, 20(2):189–201, 2009. doi: 10.1109/TNN.2008.2005601. URL <https://ieeexplore.ieee.org/document/4749258>.
- Tom Fawcett. An introduction to roc analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006. doi: 10.1016/j.patrec.2005.10.010.
- S. Ghosh and D. L. Reilly. Credit card fraud detection with a neural-network. In *Proceedings of the Twenty-Seventh Hawaii International Conference on System Sciences*, pages 621–630. IEEE, 1994. doi: 10.1109/HICSS.1994.323314.

- Addison Howard, Bernadette Bouchon-Meunier, Catherine Huang, inversion, John Lei, Lynn@Vesta, Marcus2010, and Prof. Hussein Abbass. Ieee-cis fraud detection. Kaggle, 2019. URL <https://www.kaggle.com/competitions/ieee-fraud-detection>. Dataset provided by Vesta Corporation.
- Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, pages 3146–3154, 2017.
- Fatemeh Moradi, Mehran Tarif, and MohammadHossein Homaei. Robust fraud detection with ensemble learning: A case study on the IEEE-CIS dataset. *Preprints.org*, July 2025. doi: 10.20944/preprints202507.1711.v1. Not peer-reviewed version.
- Hanchuan Peng, Fuhui Long, and Chris Ding. Feature selection based on mutual information: Criteria of max-dependency, max-relevance, and min-redundancy. *Pattern Recognition*, 38(8):1226–1238, 2005. doi: 10.1016/j.patcog.2005.01.019. URL <https://www.sciencedirect.com/science/article/pii/S0031320305001517>.
- Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. Catboost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3): e0118432, 2015. doi: 10.1371/journal.pone.0118432.
- Jorge R. Vergara and Pablo A. Estévez. A review of feature selection methods based on mutual information. *Neural Computing and Applications*, 24(1):175–186, 2014. doi: 10.1007/s00521-013-1368-0. URL <https://link.springer.com/article/10.1007/s00521-013-1368-0>.
- Wei Wei, Jinjiu Li, Longbing Cao, Yuming Ou, and Jiahang Chen. Effective detection of sophisticated online banking fraud on extremely imbalanced data. *World Wide Web*, 16(4):449–475, 2013. doi: 10.1007/s11280-012-0178-0. URL <https://doi.org/10.1007/s11280-012-0178-0>.